DM587

Scientific Programming

Lecture 06: ODEs and dynamical systems

Sai Ganesh Nagarajan

10 November 2026

Overview

Understand the model

- State variables and equations.
- Equilibria and local stability.
- Linear and nonlinear ODEs.

Run the model

- Write the derivative function.
- Supply initial values and call SciPy.
- Read the returned arrays and plot them.

ODEs

An ordinary differential equation describes how a state changes with one independent variable, usually time:

$$\boxed{\frac{dz}{dt} = f(t, z), \qquad z(t_0) = z_0.}$$

- $z(t) \in \mathbb{R}^n$: the state; $f(t, z) \in \mathbb{R}^n$: its derivatives.
- z_0: the known initial state at time t_0.
- Find the trajectory $z(t)$ that satisfies the equations and initial values.

For example, a pendulum has state $z = (\theta, \omega)$: angle and angular velocity. Its equations give their rates of change.

Notation: here $\dot{z} = dz/dt$ denotes the first time derivative, and $\ddot{z} = d^2z/dt^2$ the second.

Vector form

Collect the n state variables into one column vector:

$$z(t) = \begin{bmatrix} z_1(t) \\ \vdots \\ z_n(t) \end{bmatrix}, \qquad z(t_0) = z_0.$$

The equation $\dot{z} = f(t, z)$ expands to

$$\boxed{\begin{bmatrix} \dot{z}_1(t) \\ \vdots \\ \dot{z}_n(t) \end{bmatrix} = \begin{bmatrix} f_1(t, z_1(t), \ldots, z_n(t)) \\ \vdots \\ f_n(t, z_1(t), \ldots, z_n(t)) \end{bmatrix}.}$$

Each row is one ODE. Each f_i gives one derivative and may depend on time and all the state variables.

Types of ODE

Linear in the state

$$\dot{z} = A(t)z + b(t).$$

$A(t)$ and $b(t)$ may depend on time. $b(t)$ is a prescribed forcing term.

Example: $\dot{u} = -u + \sin t$.

General, possibly nonlinear

$$\dot{z} = f(t, z).$$

The state can enter nonlinearly.

Example: $\dot{\theta} = \omega$,
$\dot{\omega} = -\sin\theta - 0.4\omega$.

Autonomous: $f(z)$ has no explicit time dependence. The pendulum above is autonomous, although its state changes with time.

Non-autonomous: $f(t, z)$ depends explicitly on time, as in the forced example $\dot{u} = -u + \sin t$.

Solutions

A **solution** is a differentiable function $z(t)$ that satisfies

$$\dot{z}(t) = f(t, z(t)) \quad \text{at every time in its interval,} \qquad z(t_0) = z_0.$$

It gives the whole trajectory, not just one state.

Example

$$\dot{u} = -u, \qquad u(0) = 1.$$

A solution is $u(t) = e^{-t}$.

Check both conditions

$$\frac{d}{dt} e^{-t} = -e^{-t} = -u(t), \qquad u(0) = 1.$$

Under standard regularity assumptions on f (for example, f is continuously differentiable near the initial data), a unique solution exists locally around t_0.

Numerical integration approximates values of this function at selected times.

Equilibria

For an autonomous ODE, a state that stays unchanged is an **equilibrium** or fixed point z^*:

$$\dot{z} = f(z), \qquad \boxed{f(z^*) = 0.}$$

Zero derivative means no motion: if $z(0) = z^*$, then $z(t) = z^*$ for all later times.

Local asymptotic stability: sufficiently small perturbations stay close and return to the equilibrium.

We examine the displacement $\delta = z - z^*$, rather than requiring $z^* = 0$.

Jacobian

Check local stability using the Jacobian.

Here we consider an autonomous ODE $\dot{z} = f(z)$.

For two state variables, $f(z) = (f_1(z), f_2(z))^T$:

$$J(z^*) = \begin{bmatrix} \partial f_1/\partial z_1 & \partial f_1/\partial z_2 \\ \partial f_2/\partial z_1 & \partial f_2/\partial z_2 \end{bmatrix}\Bigg|_{z=z^*}.$$

Near the equilibrium, write $z = z^* + \delta$. The linear approximation is

$$\boxed{\dot{\delta} \approx J(z^*)\delta.}$$

For a linear system, the Jacobian is simply its coefficient matrix. For a nonlinear system, this approximation is local.

Eigenmodes

For the linearized system $\dot{\delta} = J\delta$, choose a real eigenpair $Jv = \lambda v$ and write the displacement along v as $\delta(t) = a(t)v$.

$$\dot{a}(t)v = J(a(t)v) = a(t)\lambda v \quad \Longrightarrow \quad \dot{a} = \lambda a.$$

This scalar ODE has solution $a(t) = c\, e^{\lambda(t-t_0)}$, where $c = a(t_0)$. Hence, along this mode,

$$\boxed{z(t) - z^* \approx c\, e^{\lambda(t-t_0)} v.}$$

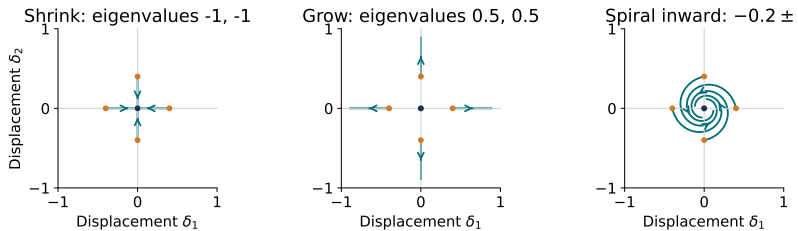
- $\lambda < 0$: the difference from z^* shrinks.
- $\lambda > 0$: a nonzero difference grows.

The formula is exact for the linearized system; it approximates the nonlinear trajectory near the equilibrium.

Eigenvalues

Along one real eigenmode, the difference from the equilibrium behaves as

$$z(t) - z^* \approx c\, e^{\lambda(t-t_0)} v.$$

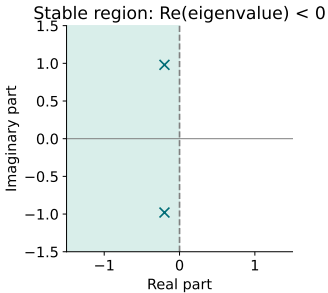


v gives a direction; λ determines growth or decay along it. Dots mark starting perturbations; arrows show forward time. The equilibrium is at the origin.

A complex pair $\lambda = \alpha \pm i\beta$ describes rotation with growth or decay: α controls the amplitude, and $|\beta|$ the angular frequency.

Stability

For a smooth autonomous ODE, evaluate $J = Df(z^*)$ at the equilibrium.



All eigenvalues have negative real parts

$$\operatorname{Re} \lambda_i < 0 \quad \text{for every } i.$$

The equilibrium is locally asymptotically stable.

If even one eigenvalue has positive real part

The equilibrium is unstable.

All real parts are ≤ 0, with at least one equal to 0

The linearization test is inconclusive.

eigvals

`numpy.linalg.eigvals(a)`

Input	a: a square $n \times n$ matrix.
Returns	One array of n eigenvalues, shape `(n,)` for one matrix.

- Eigenvalues are not necessarily sorted.
- They can be complex even when the input matrix is real.
- This function returns no eigenvectors; `np.linalg.eig` returns those too.

Signature and interface: NumPy documentation.

Eigenvalues in Python

```
import numpy as np
A = np.array([[0., 1.], [-1., -0.4]])
eigenvalues = np.linalg.eigvals(A)
print(eigenvalues)
print(np.all(np.real(eigenvalues) < 0))
```

OUTPUT

```
[-0.2+0.979796j, -0.2-0.979796j]
True
```

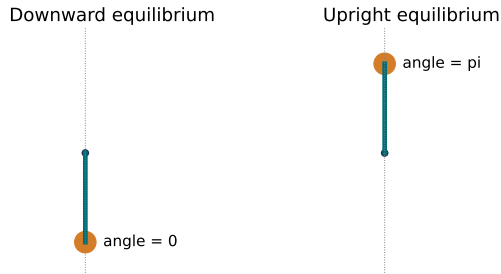
- Input: a square matrix. Output: its eigenvalues, possibly complex.
- `np.real(eigenvalues) < 0` tests each real part. `np.all(...)` returns `True` only if all entries satisfy this condition.

Pendulum

$\ddot{\theta} + 0.4\dot{\theta} + \sin\theta = 0$. Set $\omega = \dot{\theta}$ to obtain two first-order equations.

$$z = \begin{bmatrix} \theta \\ \omega \end{bmatrix}, \qquad \begin{cases} \dot{\theta} = \omega, \\ \dot{\omega} = -\sin\theta - 0.4\omega. \end{cases}$$

- θ: angle from the downward vertical.
- ω: angular velocity.
- Scaled units: gravitational coefficient 1; damping 0.4.



Equilibria: $\omega = 0$ and $\sin\theta = 0$. Inspect $(0, 0)$ and $(\pi, 0)$; copies occur every 2π.

Pendulum Jacobian

$$J(\theta, \omega) = \begin{bmatrix} 0 & 1 \\ -\cos\theta & -0.4 \end{bmatrix}.$$

Differentiate each equation with respect to each state variable.

Here J depends on angle, but not velocity.

```
def pendulum_jacobian(angle, damping):
    return np.array([
        [0., 1.],
        [-np.cos(angle), -damping]])

J = pendulum_jacobian(0., 0.4)
eigenvalues = np.linalg.eigvals(J)
```

The entries are derivatives of the equations. `eigvals` does not calculate the Jacobian or find equilibria.

Two equilibria

```
for angle_star in [0., np.pi]:
    J = pendulum_jacobian(angle_star, 0.4)
    eigenvalues = np.linalg.eigvals(J)
    print(angle_star, eigenvalues)
```

State (θ^*, ω^*)	Eigenvalues (approximately)	Local behaviour
$(0, 0)$	$-0.2 \pm 0.9798i$	Decaying oscillations
$(\pi, 0)$	$0.8198,\ -1.2198$	Unstable saddle

Each pair is (angle, angular velocity). At an equilibrium, the angle stays constant, so $\omega = \dot{\theta} = 0$. Thus $(0, 0)$ hangs downward; $(\pi, 0)$ is upright.

Initial-value problems

$$\frac{dz}{dt} = f(t, z), \qquad z(t_0) = z_0, \qquad t_0 \le t \le t_f.$$



Here t is time. The same interface can use another continuous independent variable, such as distance.

The derivative function

Write the right-hand side $f(t, z)$ for the solver to call.

$$\begin{cases} \dot{\theta} = \omega, \\ \dot{\omega} = -\sin\theta - \gamma\omega. \end{cases}$$

Input state

`[angle, velocity]`

Returned derivatives

`[angle_rate, velocity_rate]`

```
def pendulum(t, state, damping):
    angle, velocity = state
    angle_rate = velocity
    velocity_rate = (-np.sin(angle)
                     - damping * velocity)
    return [angle_rate, velocity_rate]
```

Unpack the current state, evaluate both equations, and return their rates in the same order.

We will pass `pendulum` to `solve_ivp`. The solver calls it repeatedly and uses these derivatives to advance the state. This function itself does not compute the next state.

The solver supplies `t`, even though these pendulum equations do not explicitly use time. `damping` is γ.

Derivatives

Evaluate the derivatives at time 0, angle 1 radian, angular velocity 0, and damping 0.4:

```
rates = pendulum(t=0., state=[1., 0.], damping=0.4)
print("Angular velocity:", rates[0])
print("Angular acceleration:", round(rates[1], 6))
```

OUTPUT

```
Angular velocity:  0.0
Angular acceleration:  -0.841471
```

$$\dot{\theta} = \omega = 0, \qquad \dot{\omega} = -\sin(1) - 0.4(0) \approx -0.841471.$$

The pendulum is momentarily at rest, but its angular velocity is starting to decrease. These two rates tell the solver how the state changes.

Why pass `t`? SciPy requires the interface `fun(t, state, ...)`. This pendulum is autonomous: its rates depend on the state and damping, so `t` is accepted but unused. A time-varying model would use it.

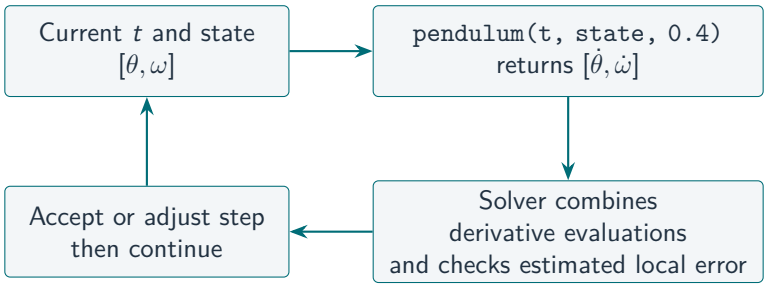
Preparing the solve

Goal: compute the pendulum's angle and angular velocity from time 0 to 20.

1. **Write the derivative function.** Our `pendulum` function returns angle rate and angular acceleration in state order.
2. **Choose the integration interval.** Start at time 0 and finish at time 20: `(0, 20)`.
3. **Specify the initial state.** At the starting time, angle is 1 radian and angular velocity is 0: `[1., 0.]`.
4. **Set the model parameters.** Use damping 0.4, passed through `args=(0.4,)`.
5. **Choose when to save the solution.** Here we request 401 output times using `t_eval`. This is optional; the solver chooses its internal steps.

Pass these inputs to `solve_ivp`, check that it succeeded, then plot the returned states against time.

Integration



One saved output point need not correspond to one function call. The solver may evaluate intermediate states many times.

Calling SciPy

```
from scipy.integrate import solve_ivp

times = np.linspace(0, 20, 401)  # 401 saved times
sol = solve_ivp(
    pendulum, (0, 20), [1., 0.],
    t_eval=times, args=(0.4,), method="RK45",
    rtol=1e-6, atol=1e-9)
```

Return the solution every 0.05 time units from 0 to 20. The solver chooses its own internal integration step sizes.

Pass **the function**, `pendulum`, without parentheses. SciPy calls it repeatedly with changing states and times.

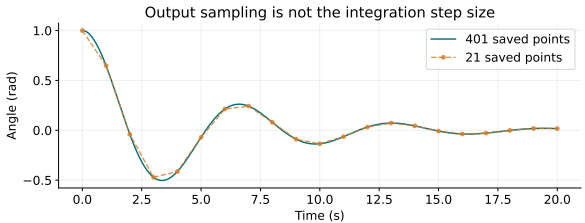
Saved points

```
coarse_times = np.linspace(
    0, 20, 21)

# Replace only the saved grid:
# t_eval=coarse_times
```

`t_eval` sets **where to save output**.

The solver still chooses its own integration steps.



More saved points can make a line look smoother without making the integration more accurate.

Tolerances

Local error scale for state entry i : $\quad \mathrm{atol} + \mathrm{rtol}\,|z_i|$.

Tolerance	SciPy defaults	Our main example
`rtol`	`1e-3`	`1e-6`
`atol`	`1e-6`	`1e-9`

- `rtol`: relative tolerance; scales the error allowance with the size of the state.
- `atol`: absolute tolerance; supplies an error allowance near zero.
- Smaller tolerances ask for more accurate local steps, often at more cost.
- This is not a guaranteed bound on the final global error.

The solution array

`sol` is a result object. For our two-state pendulum and 401 output times:

$$\texttt{sol.t.shape} = (401,), \qquad \texttt{sol.y.shape} = (2, 401).$$

	t = 0.0	t = 0.05	t = 0.1	t = 20.0
angle: sol.y[0]	1.0	0.999	0.9959	0.0172
velocity: sol.y[1]	0.0	-0.0416	-0.0824	-0.0096

Four selected columns are shown; the full array contains 401 states.

- Each **column** of `sol.y` is a state at one saved time.
- Row 0 contains angles; row 1 contains angular velocities.
- `sol.y[:, j]` matches `sol.t[j]`.

Indexing

```
angle = sol.y[0]           # all saved angles
velocity = sol.y[1]        # all saved velocities
initial = sol.y[:, 0]      # both entries at the start
last = sol.y[:, -1]        # both entries at the end
```

Expression	Shape	Meaning
`sol.t`	`(401,)`	Saved times
`sol.y[0]`	`(401,)`	Angle at those times
`sol.y[:, j]`	`(2,)`	One full state

SciPy's attribute name `y` means the state array, not a physical vertical coordinate.

Checking the result

```
print(sol.success)
print(sol.message)
if not sol.success:
    raise RuntimeError(sol.message)

print(sol.t[-1])
print(sol.nfev)
```

- `success`: did the solver finish successfully?
- `message`: explanation of how it stopped.
- `sol.t[-1]`: last stored time; here we expect 20.
- `nfev`: number of derivative-function evaluations.

Return value

`sol = solve_ivp(...)` returns an object with named attributes:

Attribute	Type / shape	Meaning
`sol.t`	`(m,)` array	The m stored independent-variable values.
`sol.y`	`(n,m)` array	n state variables at those m points.
`sol.success`	Boolean	Normal completion or a terminal event.
`sol.message`	String	Explanation of why the solver stopped.
`sol.status`	Integer	-1: failure; 0: interval end; 1: terminal event.
`sol.nfev`	Integer	Number of derivative evaluations.

If requested, `sol.sol` is an interpolator; otherwise it is `None`. Event times and states are in `sol.t_events` and `sol.y_events`.

Return fields: SciPy documentation.

Required arguments

Argument	What to supply	Our example
`fun`	A function taking `(t, state)` and returning derivatives in state order. Extra parameters follow if `args` is used.	`pendulum` returns two rates.
`t_span`	A pair `(start, end)`.	`(0, 20)`
`y0`	One-dimensional initial-state array with n entries.	`[1., 0.]`: angle and velocity.

The derivative returned by `fun` must also have n entries. The equations, initial state and return order must agree.

Optional arguments

Argument	Meaning and value in our call
`method="RK45"`	Which numerical integration method to use. We keep this default.
`t_eval=None`	Default: save solver-selected times. We supply `times` to request our output grid.
`args=None`	Default: no extra parameters. We use `(0.4,)` for damping.
`rtol`, `atol`	Error-control options. Defaults are `1e-3`, `1e-6`; we use `1e-6`, `1e-9`.

`None` means that no value was supplied for that optional feature.

Options and defaults: SciPy documentation.

Passing parameters

Our function has one extra parameter: `pendulum(t, state, damping)`.

With the comma
`args=(0.4,)`
A tuple containing one value.

SciPy unpacks it and calls
`pendulum(t, state, 0.4)`

Without the comma
`args=(0.4)`
This is just the float `0.4`. Parentheses alone do not make a tuple.

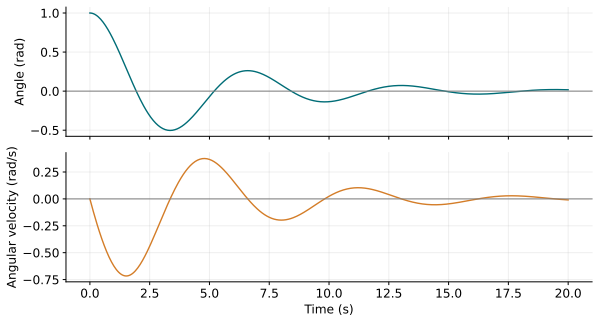
`solve_ivp` raises a `TypeError`: the number cannot be unpacked into extra arguments.

One extra parameter: use `args=(value,)`. With two extra parameters, use `args=(value1, value2)` in the function's parameter order.

Time plots

```
fig, axs = plt.subplots(
    2, 1, sharex=True)
axs[0].plot(sol.t, sol.y[0])
axs[1].plot(sol.t, sol.y[1])
axs[0].set_ylabel("Angle")
axs[1].set_ylabel("Velocity")
axs[1].set_xlabel("Time")
plt.show()
```

Horizontal coordinate: saved time.
Vertical coordinate: one state variable.

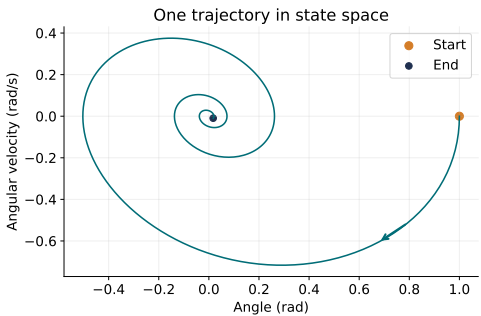


Phase space

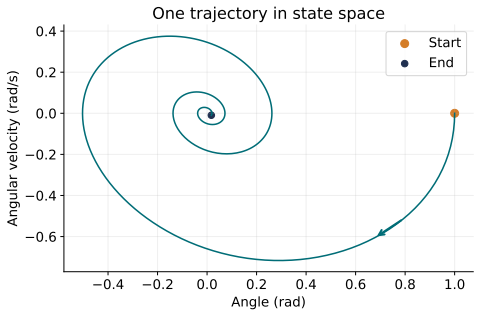
```
fig, ax = plt.subplots()
ax.plot(sol.y[0], sol.y[1])
ax.set_xlabel("Angle")
ax.set_ylabel("Velocity")
plt.show()
```

Each point pairs an angle with its velocity at the **same time**.

Time orders the path; it is not an axis.



Reading a phase plot



- Start at $(1, 0)$: displaced, initially at rest.
- The path first moves toward negative velocity.
- It crosses angle zero while still moving.
- Damping makes successive oscillations smaller.
- The state approaches $(0, 0)$.

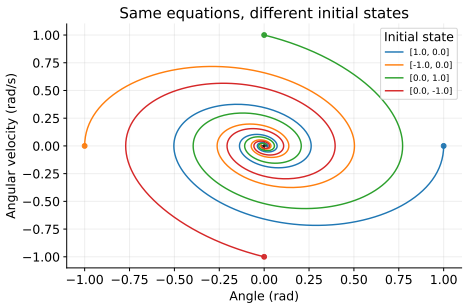
This is a path through **states**, not the pendulum bob's physical path.

Initial conditions

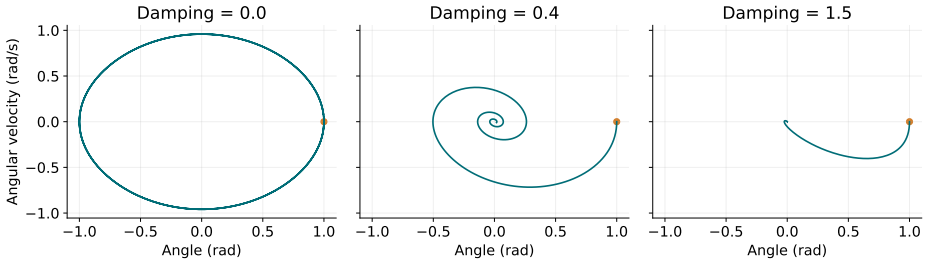
```
for initial in initial_states:
    run = solve_ivp(
        pendulum, (0, 20), initial,
        args=(0.4,), t_eval=times,
        rtol=1e-6, atol=1e-9)
    ax.plot(run.y[0], run.y[1])
```

`initial_states` contains `[1,0]`, `[-1,0]`, `[0,1]`, `[0,-1]`.

Same function and parameter; a different starting state on each call.



Damping




```
# Change this argument in the same solve_ivp call:
args=(damping_value,)
```

No damping: oscillations persist. Positive damping: energy is lost and the phase path moves inward. The equations and state order are unchanged.

Linear ODEs

$$\dot{z} = Az, \qquad A = \begin{bmatrix} 0 & 1 \\ -1 & -0.4 \end{bmatrix}.$$

```
A = np.array([[0., 1.], [-1., -0.4]])
def linear_rhs(t, state):
    return A @ state

linear_sol = solve_ivp(linear_rhs, (0, 20), [1., 0.])
```

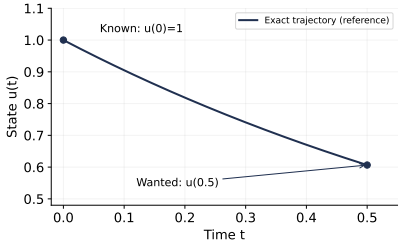
`linear_rhs` returns the two derivatives `A @ state`. Pass this function as the first argument, just as we passed `pendulum`. The solver returns a trajectory in `linear_sol`.

This is the pendulum's linear approximation near angle zero, using $\sin\theta \approx \theta$.

One step

The **Fundamental Theorem of Calculus** connects the derivative to the next state:

$$u(t+h) = u(t) + \underbrace{\int_t^{t+h} f(s, u(s))\, ds}_{\text{change over the step}}.$$



Example: $\dot{u} = -u$, $u(0) = 1$.

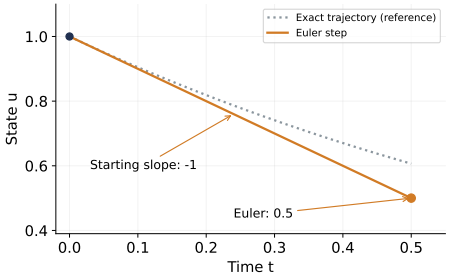
We want $u(0.5)$, so

$$u(0.5) = 1 + \int_0^{0.5} -u(s)\, ds.$$

Use a few evaluations of f to approximate the integral.

Euler

Approximate the integral using the starting slope throughout the step.



Starting slope: $f(0,1) = -1$.

With $h = 0.5$, the estimated change is

$$\int_0^{0.5} -u(s)\,ds \approx 0.5(-1) = -0.5.$$

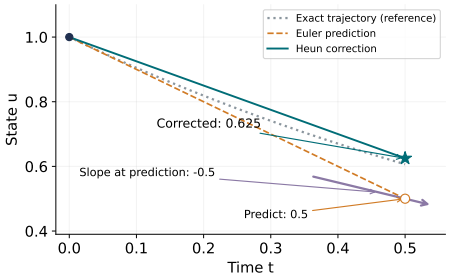
Add this to the initial state:

$$\boxed{u_{\mathrm{E}} = 1 - 0.5 = 0.5.}$$

The straight step follows the starting slope. The true slope changes, so this approximation differs from the exact endpoint, about 0.6065.

Heun

Use the starting slope and a slope at an Euler-predicted endpoint.



Predict: Euler gives $u_{\mathrm{E}} = 0.5$.

Evaluate: the slope there is $f(0.5, 0.5) = -0.5$.

Average slope:

$$\frac{-1 + (-0.5)}{2} = -0.75.$$

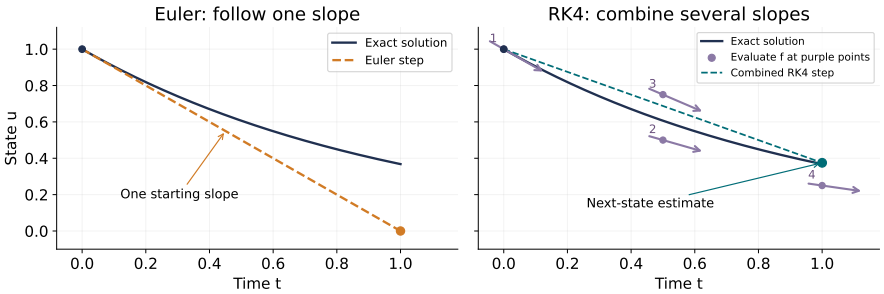
Correct from the initial state:

$$\boxed{u_{\mathrm{H}} = 1 + 0.5(-0.75) = 0.625.}$$

Heun approximates the derivative integral using two slopes. The graph shows the resulting state.

Euler and Runge–Kutta

Compare one step of length $h = 1$ for $\dot{u} = -u$, $u(0) = 1$.

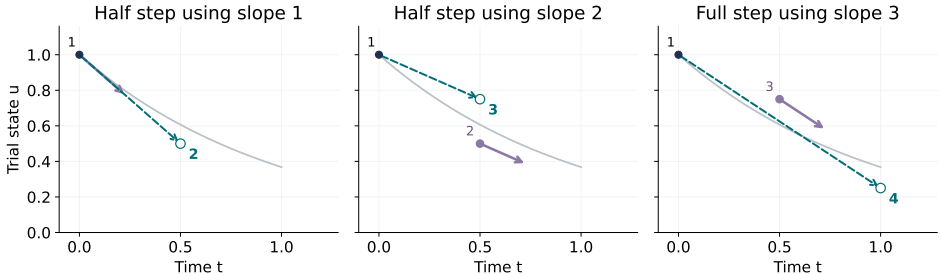


Euler: use one slope. **RK4:** evaluate $f(t, u)$ at the four purple trial points; their arrows show the returned slopes. Trial points can lie off the exact curve.

Next value: Euler 0; RK4 0.375; exact $e^{-1} \approx 0.367879$. The notebook contains the calculation.

RK4 trial states

Each trial starts from the **original state**. Use a slope already computed to place the next trial point. Here $\dot{u} = -u$, $u(0) = 1$, $h = 1$.

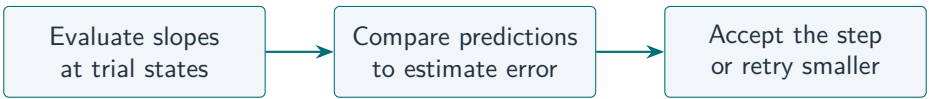


Purple arrows: slopes evaluated at the numbered trial states. **Teal arrows:** use that slope from the original state to construct the next trial.

Two midpoint trials improve the slope estimate; the endpoint trial adds information from the end of the step. Their weighted average gives the final RK4 step, not any one trial point.

RK45

SciPy's default method uses the same broad idea: evaluate several trial slopes to predict the next state.

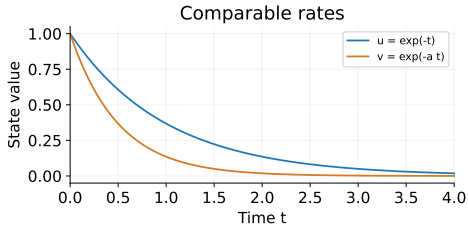


- The error estimate and tolerances guide the internal step size.
- The detailed RK45 coefficients are handled by SciPy.
- The saved output times are specified separately by `t_eval`.

Stiffness

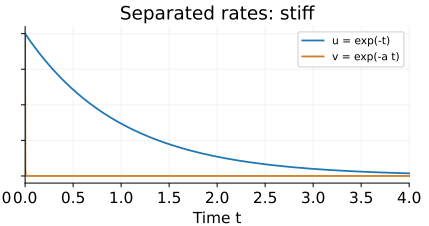
Comparable decay rates

$$\dot{u} = -u, \qquad \dot{v} = -2v.$$



Widely separated decay rates

$$\dot{u} = -u, \qquad \dot{v} = -1000v.$$



Both start at $(u, v) = (1, 1)$. On $0 \le t \le 4$, the second system is stiff: v decays on a scale of 0.001, while u changes on a scale of 1.

The fast component becomes almost zero, but it can still restrict an explicit solver's step size.

Step size

For one decay component $\dot{v} = -av$, Euler gives

$$v_{k+1} = (1 - ah)v_k.$$

To make numerical deviations shrink, require

$$|1 - ah| < 1 \quad \Longrightarrow \quad \boxed{0 < h < 2/a.}$$

System	Fastest decay rate	Euler stability restriction
$\dot{u} = -u,\ \dot{v} = -2v$	$a = 2$	$h < 1$
$\dot{u} = -u,\ \dot{v} = -1000v$	$a = 1000$	$h < 0.002$

For the stiff system, $h = 0.01$ gives multiplier $1 - 1000h = -9$: numerical deviations grow, although the exact component decays.

This bound is for Euler; RK45 has a different stability limit. Stiff solvers such as `Radau` or `BDF` can avoid this severe explicit-step restriction. Accuracy still matters.

Choosing a method

`method`	When to consider it
`"RK45"`	Default for non-stiff problems; suitable for our pendulum.
`"RK23"`	Lower-order alternative to compare at modest accuracy.
`"DOP853"`	Higher-order method for high accuracy and tight tolerances.
`"Radau"`, `"BDF"`	For stiff equations: explicit methods may need tiny stable steps.
`"LSODA"`	Detects stiffness and switches methods automatically.

If `RK45` takes unusually many steps or fails, check the model and consider a stiff solver.

Options and guidance: SciPy documentation. Select with, for example, `method="DOP853"`.

Time-varying ODEs

$$\dot{u} = -u + \sin t, \qquad u(0) = 0.$$

```
def forced_decay(t, state):
    return -state + np.sin(t)

forced = solve_ivp(forced_decay, (0, 10), [0.],
                   t_eval=np.linspace(0, 10, 201))
```

The same interface works: the solver supplies the current `t` and state to our function. Here `np.sin(t)` makes the equation non-autonomous.

Euler and Runge–Kutta methods evaluate $f(t, z)$ at the appropriate times and states; they do not require an autonomous equation.

The fixed-point eigenvalue test shown earlier is for autonomous systems. A time-varying Jacobian generally requires a different stability analysis.

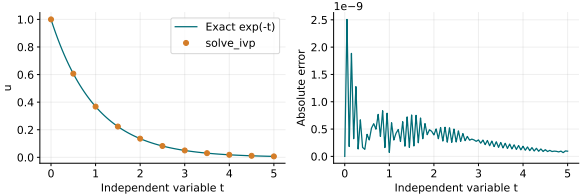
A known solution

$$\dot{u} = -u, \quad u(0) = 1.$$

$$u(t) = e^{-t}.$$

```
def decay(t, state):
    return -state

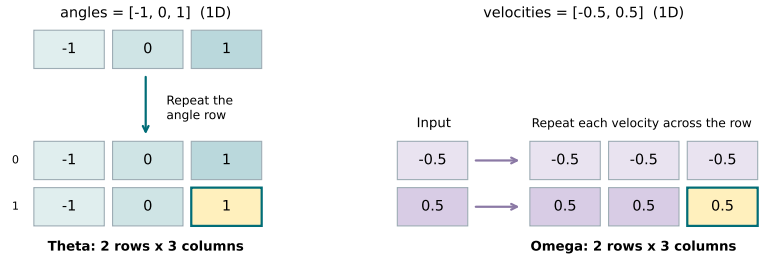
check = solve_ivp(
    decay, (0, 5), [1.],
    t_eval=np.linspace(0, 5, 101),
    rtol=1e-8, atol=1e-10)
exact = np.exp(-check.t)
```



Even a scalar ODE uses an initial-state array. Compare `check.y[0]` with `exact` to measure numerical error.

meshgrid

```
Theta, Omega = np.meshgrid(angles, velocities)
```



Both outputs have shape `(2,3)`: two velocity rows and three angle columns.

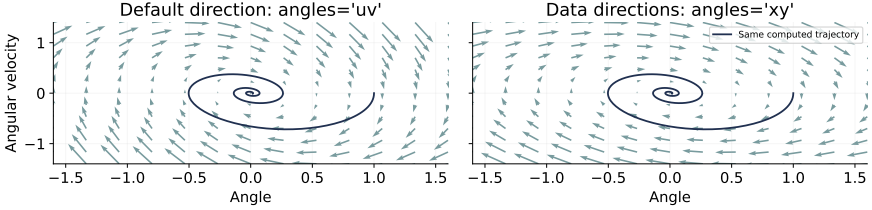
Read the highlighted entries together: `Theta[1,2]=1` and `Omega[1,2]=0.5` give the state $(\theta, \omega) = (1, 0.5)$.

Vector fields

```
angle_grid = np.linspace(-1.6, 1.6, 13)
velocity_grid = np.linspace(-1.4, 1.4, 13)
Theta, Omega = np.meshgrid(
    angle_grid, velocity_grid)
dtheta = Omega
domega = -np.sin(Theta) - 0.4 * Omega
```

```
# Default direction: angles="uv"
ax.quiver(Theta, Omega, dtheta, domega,
          scale_units="xy", scale=7)

# Directions follow the data axes
ax.quiver(Theta, Omega, dtheta, domega,
          angles="xy",
          scale_units="xy", scale=7)
```



Same states, derivatives and trajectory; only `angles` changes. Axis scales are deliberately unequal. `"xy"` keeps arrows aligned with the trajectory; `"uv"` uses screen directions.

quiver

```
ax.quiver(X, Y, U, V,
          angles="xy", scale_units="xy", scale=1)
```

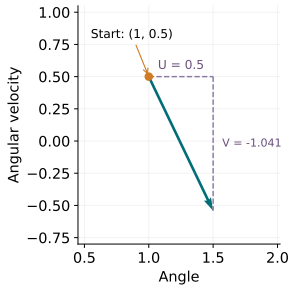
Where does each arrow start?

`X, Y`: horizontal and vertical positions.

Which way does it point?

`U, V`: horizontal and vertical vector components.

For our pendulum: positions are `Theta, Omega`; components are `dtheta, domega`.



`angles="xy"` uses the data coordinates. With `scale_units="xy"`, `scale=1` shows the supplied vector lengths; `scale=7` makes arrows seven times shorter. An arrow shows the derivative direction, not a completed integration step.

solve_ivp

The documented call signature is:

```
solve_ivp(fun, t_span, y0, method="RK45",
          t_eval=None, dense_output=False,
          events=None, vectorized=False,
          args=None, **options)
```

- `fun`, `t_span` and `y0` are required.
- Arguments with = have defaults; specify the ones you want to change.
- `**options` accepts solver settings such as `rtol`, `atol` and `max_step` as named arguments.

The return value is one result object. We name it `sol` in our example.

Signature: SciPy documentation.

Other options

These can stay at their defaults for the main example.

Option	Purpose
`dense_output=False`	With `True`, also return an interpolator for evaluating the computed solution between saved points.
`events=None`	Optional functions for detecting zero crossings; an event can be configured to stop integration.
`vectorized=False`	Keep the ordinary one-state-vector callback. `True` requires support for multiple state columns in the callback.
`max_step=np.inf`	An optional cap on internal step size; different from the output grid `t_eval`.

The notebook includes a short `dense_output=True` example. No event functions are needed for our pendulum simulation.

Checks

Code checks

- Derivative order matches state order.
- Initial state and parameters are correct.
- Solver succeeded and reached the intended endpoint.
- Array rows and plot axes are labelled.

Mathematical checks

- Is the observed behaviour plausible?
- Does it agree with local stability?
- Does tightening tolerances change the result?
- Is there an exact solution or conservation law to compare with?

Try-it notebook

Change one part of the model at a time.

- Initial state: `[1,0]` versus `[0,1]`.
- Damping: 0, 0.4, 1.5.
- Replace $\sin\theta$ by θ near the downward equilibrium.
- Try another ODE: write its derivative function, solve it, and label the plots.

The companion contains all examples, saved outputs, and the code used to generate the figures in these slides. No data download is required.

Readings

- Boyd and Vandenberghe, *Introduction to Applied Linear Algebra*, Chapter 9: Linear dynamical systems — linear dynamical systems.
- SciPy: `solve_ivp` — arguments, returned arrays and solver options.
- NumPy: `linalg.eigvals` — eigenvalues of a square matrix.
- Hank Yang, *Optimal Control and Estimation*, §5.1.2 — local stability by linearization; pendulum example.

Companion: Class 6 try-it notebook, plus an HTML copy with outputs.